

Republic of Tunisia

IT370 Course Project Report

Tunisia Heritage Quest

Android Educational Quiz Application

Student:	Mohamed Cherif Khcherif
Professor:	Najet Boughanmi
Course:	IT370
Submission Date:	May 6, 2026

Academic Year 2025–2026

Contents

Abstract	1
1 Introduction	1
2 Project Objectives	1
3 Application Overview	2
4 Technical Stack	2
5 System Architecture	2
6 User Interface and User Experience	3
7 Core Features	3
8 Implementation Notes	3
9 Testing	4
9.1 Testing Strategy	4
9.2 Validation Performed	4
9.3 Testing Evidence	4
10 Use of Artificial Intelligence	5
10.1 Tasks Assisted by AI	5
10.2 Tasks Led by Human Work	6
11 Challenges and Solutions	6
12 Conclusion	6

Abstract

This report presents *Tunisia Heritage Quest*, an Android educational quiz application developed for the IT370 course. The project was designed to promote knowledge of Tunisian cultural and historical heritage through an interactive mobile experience. The application combines a structured quiz flow, category-based exploration, difficulty settings, persistent player statistics, and a themed user interface inspired by Tunisian visual identity. The implementation was built with Kotlin, Jetpack Compose, Room, DataStore, and Navigation Compose. This report summarizes the project objectives, architecture, implementation choices, testing process, and the controlled use of artificial intelligence during development.

1 Introduction

Mobile applications are an effective medium for educational engagement because they combine accessibility, interactivity, and immediate feedback. In this project, the goal was to design and implement a quiz-based Android application centered on Tunisian heritage. The app encourages users to discover monuments, cities, and cultural sites while answering multiple-choice questions supported by visual content.

The project was also an opportunity to apply modern Android development practices, including declarative UI with Jetpack Compose, state management through ViewModels, persistent local storage, and structured navigation. Beyond functionality, the project emphasized visual quality and a culturally coherent user experience.

2 Project Objectives

The main objectives of the project were the following:

- Develop a functional Android quiz application using Kotlin.
- Organize the app with a clean architecture based on MVVM principles.
- Provide a multi-screen user flow including splash, menu, category selection, difficulty selection, quiz gameplay, and results.
- Store user settings and performance statistics locally.
- Design an interface that reflects Tunisian heritage through color, ornament, and composition.
- Deliver a project that is testable, maintainable, and appropriate for an academic software engineering context.

3 Application Overview

The final application is a single-player heritage quiz game. The player starts from a splash screen and then navigates through the main menu to select a category and difficulty level. During gameplay, each question presents an image and four possible answers. The user selects an answer, receives feedback, and proceeds through the quiz until reaching the results screen.

The implemented categories include Roman heritage, Islamic heritage, Punic and pre-Roman heritage, modern heritage, natural and mixed sites, and Tunisian cities. Different difficulty levels adjust the challenge and timer behavior. The application also includes settings such as timer enablement, sound effects, and haptic feedback.

4 Technical Stack

The project was implemented with the following technologies:

Component	Technology Used
Programming language	Kotlin
UI toolkit	Jetpack Compose with Material 3 foundations
Architecture	MVVM with unidirectional state flow
Navigation	Navigation Compose
Persistence	Room database and DataStore Preferences
Asynchronous programming	Kotlin Coroutines and Flow
Image loading	Coil with bundled local assets and remote fallback
Testing	JUnit-based unit tests and ViewModel-focused tests

5 System Architecture

The application follows a layered architecture centered on the MVVM pattern. The user interface is built with stateless or mostly stateless composables. Business and screen logic are handled inside ViewModels, which expose state to the UI through reactive flows. Repositories act as an abstraction between the ViewModels and the data sources. Local data persistence is handled through Room for player statistics and DataStore for user settings.

This structure improves maintainability by separating concerns. UI code remains focused on presentation, while data access and application logic are isolated in dedicated layers. This approach also makes the project easier to test and extend.

6 User Interface and User Experience

One of the most important aspects of this project was the creation of a strong visual identity. The interface was designed around a Mediterranean blue palette with gold accents, ornamental dividers, arched motifs, and heritage-inspired card layouts. The goal was not only to make the application attractive, but also to make its appearance consistent with the cultural theme of Tunisian architecture and historical landmarks.

The screens were redesigned to move away from a default Material appearance and toward a more intentional, themed presentation. Human design decisions played the main role in this process. The high-level structure of the app, the visual style, the screen hierarchy, and the UI/UX direction were defined manually. This includes the splash composition, main menu layout, category cards, difficulty cards, quiz screen balance, and results page styling.

7 Core Features

The project includes the following major features:

- Splash screen with thematic branding.
- Main menu with progress summary and quiz entry point.
- Category selection for different types of Tunisian heritage.
- Difficulty selection with gameplay settings.
- Quiz flow with timed questions and answer validation.
- Result screen with score summary and replay support.
- Local persistence for settings and statistics.
- Haptic feedback and sound effects controlled by user preferences.
- Local image asset support for selected questions to improve reliability.

8 Implementation Notes

Several practical engineering decisions were made during development. The application uses a single-activity model with navigation handled by a `Compose NavHost`. Player settings are stored with `DataStore`, while progress-related values such as quiz statistics are stored in `Room`. `ViewModels` manage the quiz session, answer progression, timer state, and results logic.

Because online image loading was not reliable enough for delivery conditions, the project was adapted to support bundled image assets for questions. The loader first checks the packaged local assets and only uses a remote fallback when a local image is not available. This change increased reliability and made the app more suitable for demonstration.

9 Testing

Testing was treated as an important part of the project rather than an optional final step. The objective was to verify both correctness and stability after major UI and logic changes.

9.1 Testing Strategy

The testing effort focused on the following levels:

- **Unit tests:** used to validate isolated logic and output behavior.
- **ViewModel tests:** used to check state updates, quiz progression, result calculations, and event flow.
- **Build verification:** used to ensure that the application compiles successfully after feature work and refactoring.

9.2 Validation Performed

During development, the following verification tasks were used repeatedly:

- Debug build generation through Gradle.
- Unit test execution for logic and ViewModel behavior.
- Manual on-device checking of navigation flow, themed layouts, button behavior, and image display.

These tests were especially useful after changes to quiz timing, result navigation, button responsiveness, settings toggles, and local image loading.

9.3 Testing Evidence

To support the validation process, terminal screenshots were collected as evidence of successful verification steps. The first screenshot documents successful debug build gen-

eration, the second documents successful unit test execution, and the third shows an additional successful verification run from the development environment.

```
BUILD SUCCESSFUL in 36s
38 actionable tasks: 38 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.1.0/userguide/configuration_cache_enabling.html
(base) PS C:\Users\wwwkh\AndroidStudioProjects\mohamedcherifkhcherif>
```

Figure 1: Terminal output showing successful project build verification.

```
(base) PS C:\Users\wwwkh\AndroidStudioProjects\mohamedcherifkhcherif> Get-ChildItem "app\build\outputs\apk\debug"

Répertoire : C:\Users\wwwkh\AndroidStudioProjects\mohamedcherifkhcherif\app\build\outputs\apk\debug

Mode                LastWriteTime         Length Name
----                -
-a----           06/05/2026      12:36      21691602 app-debug.apk
-a----           06/05/2026      12:36         407 output-metadata.json
```

Figure 2: Terminal output showing successful unit test execution.

```
Task :app:kspDebugKotlin
[!ksp] Schema export directory was not provided to the annotation processor so Room cannot export the schema. You can either provide 'room.schemaLocation' annotation processor argument by applying the Room Gradle plugin (id 'androidx.room') OR set exportSchema to false.

Task :app:compileDebugKotlin
file:///C:/Users/wwwkh/AndroidStudioProjects/mohamedcherifkhcherif/app/src/main/java/com/example/heritagequest/domain/model/Category.kt:8:5 This annotation is currently applied to the value parameter only, but in the future it will also be applied to field.
To opt in to applying to both value parameter and field, add '-Xannotation-default-target=param-property' to your compiler arguments.
To keep applying to the value parameter only, use the '@param:' annotation target.
See https://youtrack.jetbrains.com/issue/KI-73255 for more details.
file:///C:/Users/wwwkh/AndroidStudioProjects/mohamedcherifkhcherif/app/src/main/java/com/example/heritagequest/ui/screens/MenuScreen.kt:100:47 'val Icons.Outlined.ArrowForward: ImageVector' is deprecated. Use the AutoMirrored version at Icons.AutoMirrored.Outlined.ArrowForward.
===== 100% EXECUTING [49s]

BUILD SUCCESSFUL in 52s
38 actionable tasks: 38 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.1.0/userguide/configuration_cache_enabling.html
(base) PS C:\Users\wwwkh\AndroidStudioProjects\mohamedcherifkhcherif>
```

Figure 3: Additional recorded testing evidence used during final validation.

10 Use of Artificial Intelligence

Artificial intelligence was used in a limited and controlled manner during this project. Its role was supportive rather than dominant.

10.1 Tasks Assisted by AI

AI was used for:

- question generation,
- code review,
- test generation and testing support,

- image generation,
- and a small portion of code implementation.

10.2 Tasks Led by Human Work

The main human contribution remained central to the project. Human work covered:

- the high-level design of the application,
- feature direction and product decisions,
- the UI/UX design vision,
- visual style selection,
- review of implementation quality,
- and final integration decisions.

In other words, AI was used as an assistant for acceleration and support, while the human role remained responsible for the conceptual direction, aesthetic choices, and major product decisions.

11 Challenges and Solutions

The project encountered several practical challenges. One challenge was the mismatch between the initial functional implementation and the desired UI quality. This was solved by introducing a custom themed design system and redesigning the main screens. Another challenge involved quiz image reliability. Remote loading alone was not stable enough, so the solution was to support bundled local assets for selected quiz questions. A further issue concerned responsiveness on smaller layouts, especially for action buttons, which was addressed by revising component sizing and layout behavior.

These iterations improved both usability and presentation while keeping the application aligned with the original educational objective.

12 Conclusion

Tunisia Heritage Quest demonstrates how modern Android development tools can be used to build an educational application with both technical structure and strong visual identity. The project integrates Kotlin, Jetpack Compose, ViewModels, Room, DataStore, and testing practices into a coherent mobile product. It also reflects a balanced

workflow in which AI was used selectively for support, while the main design and UX direction remained human-driven.

Overall, the project satisfies the academic goal of producing a functional, themed, and structured Android application while offering room for future extension, such as adding more bundled images, broader test coverage, and expanded question content.